

Elaborato Assembly
Laboratorio di Architettura degli Elaboratori

Hamdane, Walid Segala, Tommaso
VR500517 VR464649

Anno Accademico 2023/2024

Contents

1	Specifiche	3
1.1	Struttura programma	3
2	File e funzioni	3
2.1	pianificatore.s	3
2.1.1	Variabili	4
2.2	EDF e HPF	4
2.2.1	Variabili	4
2.3	task.s	5
2.3.1	Variabili	5
2.4	atoi e PrintInt	5
3	Test e scelte	5

1 Specifiche

Viene chiesto di sviluppare in assembly un software per la pianificazione dei lavori di un sistema di produzione. Il programma legge da file **massimo** 10 prodotti, che produrrà nelle successive 100 unità temporali, lavorando un prodotto per unità temporale. Ogni prodotto è identificato da 4 numeri interi:

- **ID** : Codice identificativo, compreso tra 1 e 127.
- **Durata** : Durata in unità di tempo, tra 1 e 10.
- **Scadenza** : Tempo entro il quale deve finire la produzione, valore da 1 a 100.
- **Priorità** : Indicatore priorità produzione, va da 1 a 5.

Bisogna anche calcolare quella che è la penalità in caso di produzione che supera la scadenza, questa è calcolata nella seguente maniera:

$$Penalita = Ritardo * priorit$$

1.1 Struttura programma

Il codice in breve deve :

1. Aprire file di input e file output con syscall 5
2. Stampare un messaggio e attendere input utente
3. eseguire algoritmo scelto
4. stampare a video e su file output le statistiche nel formato

ID : INIZIO

5. stampare inoltre la somma delle penalità

2 File e funzioni

2.1 pianificatore.s

File principale che contiene *_start* prende dallo stack il path del file, lo apre con la syscall 5 e salva il file descriptor nella variabile *FileI*, controlla *eax* per vedere se si è aperto correttamente in caso contrario stampa un errore ed esce.

Una volta aperto il file stampa a video le scelte e con la syscall 3 legge un input da tastiera, anche qui da errore in caso di input che non sia 1,2,3, 1 chiama EDF, 2 HPF e il 3 chiude file e programma.

2.1.1 Variabili

- FileI, int, qui viene salvato il file descriptor del file input
- NoParam, ascii, messaggio di errore di caricamento del parametro
- FileL_err ascii, messaggio di errore apertura file input
- menu_msg, ascii, messaggio del menu che presenta all'utente le scelte disponibili
- scelta_err_msg ascii, messaggio di scelta sbagliata
- input, .space nella sezione .bss, spazio di 1 byte non allocato per il carattere di input

2.2 EDF e HPF

Dopo aver letto l'input si confronta il valore in ascii e si chiama una delle due.

Partono ripristinando lseek con la syscall 19, questo permette di leggere il file in un loop senza necessita di riavviare il programma.

Il file viene letto byte per byte, quando si incontra una virgola o un newline si salva nello stack il numero letto precedentemente, per convertire la stringa si chiama la funzione **atoi**, viene tenuto conto dei valori letti con una variabile **"valori"**, e le righe con la variabile righe.

Ad ogni newline viene controllata l'ultima riga letta, per assicurarsi che i valori siano in linea con quelli delle specifiche, se è presente un errore si da errore e si torna alla funzione chiamante.

Quando si arriva all'EOF controllo che lines ovvero il numeor di prodotti letti non superi 10, in caso da errore, altrimenti avanzo con l'ordinamento.

Qui cambiano leggermente le due funzioni EDF e HPF, spiego il funzionamento di EDF, ma per HPF l'unica differenza è nelle colonne confrontate e nella condizione per cui si scambiano due colonne (HPF si fa un sorting decrescente, ovvero ad ogni ciclo il numero più basso di quelli letti viene spostato in fondo).

Si parte dalla prima riga usando il numero di valori letti, salvato in edx per muoversi nello stack, se leggo 40 valori in EDF devo sottrarre 3 per posizionarmi sulla colonna delle scadenze, letta la deadline della riga A, faccio di nuovo *subl\$4, %edx* per leggere quella della riga B, confronto questi due valori e se $A > B$ chiamo la funzione **Sorter**, se $A = B$ valuto le priorità più alte, altrimenti avanzo.

Ho un counter delle righe che decremento ad ogni confronto, se 0 allora ho finito il ciclo, tengo anche conto degli scambi nel ciclo ocn un flag in %ecx che se a fine ciclo rimane 0 allora non ci sono stati scambi e quindi lo stack è ordinato, altrimenti riparto dalla prima riga.

Ordinato lo stack faccio call **task**

2.2.1 Variabili

- lines linee lette
- valori, elementi letti
- buffer, contiene il carattere letto

2.3 task.s

Svolge la funzione di stampa, stampa, gli ID, usa %ecx come "orologio" parte da 0, e ad ogni stampa prima il tempo di inizio, poi somma la DURATA del prodotto per aggiornare %ecx, confronta poi %ecx con la scadenza del prodotto e in caso di ritardo calcola e incrementa la penalità. Finiti i prodotti, ovvero quando edx=0, stampa il tempo di conclusione e la penalità totale.

Usala funzione **PrintInt** per stampare gli interi.

2.3.1 Variabili

- lines linee lette
- valori, elementi letti
- penalty, contiene la penalità

2.4 atoi e PrintInt

Atoi è una semplice funzione che presa una stringa la converte in intero, PrintInt al contrario prende un intero e restituisce una stringa che stampa.

3 Test e scelte

Nella directory ordini sono presenti i 3 file HPF.txt, EDF.txt e None.txt, il primo da penalty 0 con l'algoritmo HPF, il secondo da lo stesso risultato quando si usa EDF, il terzo da con entrambi.

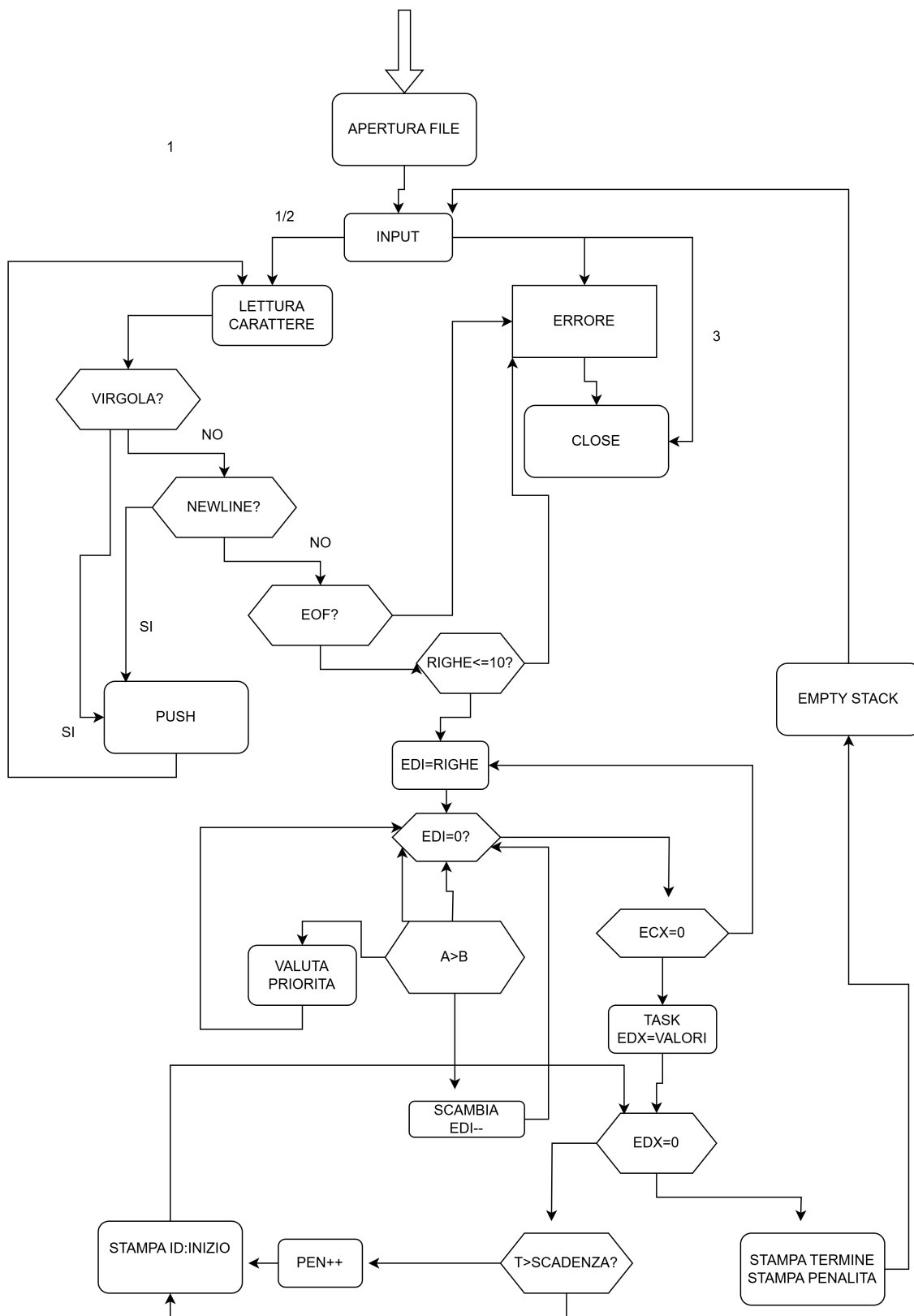


Figure 1: Diagramma molto semplificato delle funzioni base